Complete SQL Project Requirements, Flow & Implementation Guide

Banking Data Analysis — MySQL Capstone Project

1. Project Overview

A complete final SQL project in which you build and analyze a simulated banking database. It is designed to exercise the full syllabus: DDL, DML, DQL, constraints, joins, aggregation, functions, CASE, subqueries, window functions, CTEs, views, data cleaning, indexes, EXPLAIN and transactions.

2. Business Scenario

You are a junior SQL/Data Analyst at a bank. The bank needs a relational system for customers, branches, accounts, beneficiaries, transactions, loans, loan payments, cards and employees. Management wants analytical answers about customer activity, branch performance, transaction trends, loan risk and high-value customers.

3. Objectives

- Design a relational banking database.
- Create tables with suitable data types and constraints.
- Import and validate the supplied CSV data.
- Write basic and advanced SQL queries.
- Use joins, aggregation, subqueries, CTEs and window functions.
- Clean and validate data using SQL.
- Create reusable views.
- Use indexes and EXPLAIN for performance analysis.
- Demonstrate COMMIT, ROLLBACK, SAVEPOINT and ACID concepts.
- Present business insights supported by SQL.

4. Technology & Dataset

Item	Requirement
Database	MySQL 8.0+ / MariaDB 10.11+
Tool	MySQL Workbench recommended
Database name	banking_system
Input	9 CSV files
Records	6,400+ synthetic records
Output	SQL scripts + ER diagram + final report

VRajasha

5. Dataset Files

File	Rows	Purpose
customers.csv	500	Customer master (includes a few intentional duplicate/missing values for the cleaning module)
branches.csv	30	Branch master
accounts.csv	586	Customer accounts
beneficiaries.csv	401	Saved beneficiaries for transfers
transactions.csv	3,000	Transaction history (includes a few negative/suspicious amounts)
loans.csv	360	Loans
loan_payments.csv	941	Loan repayment history
cards.csv	443	Debit/credit cards
employees.csv	150	Employees and manager hierarchy (self-referencing)

6. Database Relationship Flow

- customers → accounts → transactions
- customers → beneficiaries
- customers → loans → loan_payments
- customers → cards
- branches → accounts
- branches → loans
- branches → employees
- employees → employees through manager_id (SELF JOIN / Recursive CTE)

Students should draw an ER diagram before writing CREATE TABLE statements.

7. Table Responsibilities

Table	Purpose	Keys
customers	Customer master	customer_id PK
branches	Branch master	branch_id PK
accounts	Bank accounts	account_id PK; customer_id, branch_id FK
beneficiaries	Saved beneficiaries	beneficiary_id PK; customer_id FK
transactions	Transaction history	transaction_id PK; account_id FK
loans	Loan master	loan_id PK; customer_id, branch_id FK
loan_payments	Loan repayment history	payment_id PK; loan_id FK
cards	Card information	card_id PK; customer_id, account_id FK
employees	Employee hierarchy	employee_id PK; branch_id FK; manager_id self-reference

VRajasha

8. DDL Requirements

- Create the banking_system database.
- Create all nine tables.
- Use suitable MySQL data types.
- Apply PRIMARY KEY and FOREIGN KEY relationships.
- Use NOT NULL, UNIQUE, DEFAULT and CHECK where appropriate.
- Use AUTO_INCREMENT for surrogate keys.
- Demonstrate ALTER TABLE.
- Demonstrate DELETE vs TRUNCATE vs DROP on a safe practice table.

9. Data Loading & Validation

- Load parent tables before child tables (branches/customers before accounts, accounts before transactions/cards, loans before loan_payments).
- Verify expected row counts after each import.
- Check for duplicate primary keys.
- Check foreign-key validity.
- Check important NULL values (e.g., missing phone numbers).
- Document import errors and fixes.

11. SQL

Write one query for each of the following:

1. List the number of customers living in each city, ordered from most to fewest.
2. Find the total balance held in accounts, grouped by account_type.
3. Find branches that manage more than 15 accounts.
4. For every branch, show how many accounts it holds and the branch's total balance (include branches with zero accounts).
5. Find customers who do not currently hold any account.
6. For every employee, show their name, designation, and their manager's name (if they have one).
7. Categorize every account into a balance_band of 'Low' (under ₹10,000), 'Medium' (₹10,000–100,000) or 'High' (over ₹100,000).
8. Find the single customer who has spent the most in total transaction amount.
9. Find employees who earn more than the average salary of their own branch.
10. Find the top 3 customers by account balance in every city.
11. For each month, show the total transaction amount and compare it with the previous month's total.
12. Create a view called customer_account_summary showing each customer's total number of accounts and total balance.
13. Using a recursive CTE, produce the full employee reporting hierarchy starting from each Regional Head.
14. identify duplicate customer records based on a normalized (trimmed, uppercased) email address.
15. Identify any transactions with a negative amount.

VRajasha

16. Use EXPLAIN to see how MySQL executes a lookup of transactions for a specific account_id, then add an index and compare the plan.
17. Branches with the most overdue/defaulted loans?
18. Write a transaction that transfers ₹5,000 from one account to another, ensuring both the debit and the credit succeed together or not at all.
19. Add a SAVEPOINT before the transfer and show a ROLLBACK TO SAVEPOINT that undoes only the transfer, leaving any earlier work in the same session untouched.
20. Explain, in your own words, how your transaction satisfies each of the four ACID properties: Atomicity, Consistency, Isolation and Durability.

VRajasha